

Revue de US-008 — Saisir une journée travaillée sur une mission

Revue faite le 2026-08-28 par l'agent `business-analyst`, avant toute conception. Ce document juge la **demande**, pas la solution. Aucun fichier de `specs/` n'a été modifié.

Verdict

À clarifier — le cas nominal est net et borné, mais trois refus annoncés sans code ni message, un état vide non décrit et un CRA mensuel que personne ne crée obligerait le développeur à deviner ; chaque question ci-dessous se referme en une phrase.

	Bloquants	À clarifier	Suggestions
Nombre	0	5	3

Les six axes

Axe	Jugement	Justification
Compréhensible	OK	Le vocabulaire est celui du README (consultant, mission, affectation, jour ouvré) et le rôle est nommé dès la première ligne. Deux formulations restent à deux lectures — « le nom de la mission » et « se distingue visuellement » — mais elles gênent la rédaction du test, pas la compréhension du besoin.
Unitaire	OK	Une seule fonctionnalité : imputer un jour sur une mission. La suppression décrite au quatrième critère n'est pas une seconde fonctionnalité, c'est le cycle de vie de l'objet créé au deuxième — la livrer sans elle laisserait une saisie irréversible. Un seul écran, celui du calendrier de US-007.
Testable	Non	Trois critères n'ont pas de résultat observable. « La saisie est impossible sur un week-end ou un jour férié » ne dit ni le code HTTP ni le message ; « Un consultant ne peut pas saisir sur le CRA d'un autre consultant » porte sur une route qui ne nomme aucun consultant ; « se distingue visuellement d'un jour vide » n'est pas vérifiable mécaniquement.
Complète	Non	Le chemin nominal et deux cas d'erreur (jour complet, mission inaccessible) sont traités. Manquent l'état vide — aucune mission déclarable à cette date, cas que <code>list_available_missions</code> renvoie déjà comme liste vide — la valeur limite du jour futur, et le message de chaque refus.
Bornée	OK	Aucun renvoi vers une story ultérieure ; le champ <code>fraction</code> et les types d'absence sont laissés à US-009 et US-010, ce qui est le bon découpage. Les deux prérequis annoncés existent. Réserve : la notion de mission « active » vient de US-003 et US-006, non déclarés — mais numériquement antérieurs et déjà tranchés dans le code.
Implémentable	Non	Les routes et deux codes de retour sont donnés, mais le corps de la requête n'est pas nommé, aucun libellé français n'est fourni alors que l'écran en affiche au moins trois (sélecteur, jour saisi, refus), et rien ne dit qui crée l'enregistrement <code>Cra</code> du mois auquel les entrées se rattachent — aucun modèle <code>Cra</code> n'existe aujourd'hui dans <code>backend/app/models/</code> .

Constats

[À CLARIFIER] Le CRA du mois n'est créé par aucune story

`POST /api/cra/{annee}/{mois}/entrees` → 201, `DELETE /api/cra/{annee}/{mois}/entrees/{id}` → 204.

Ce qui manque : l'entrée se rattache à un CRA (consultant, année, mois, statut), mais aucune story ne dit qui le crée ni quand. US-007, prérequis déclaré, n'expose qu'un `GET` et précise que les entrées sont « vide à ce stade » — une lecture, pas une création. Côté code, `backend/app/models/` ne contient aujourd'hui que `user`, `mission` et `assignment` : il n'y a pas d'enregistrement `Cra`. Le statut initial « Brouillon » est bien défini (`CraStatus` dans `backend/app/models/enums.py`), mais l'instant de la création ne l'est pas.

Question à poser : le CRA du mois est-il créé automatiquement, en statut « Brouillon », au moment de la première saisie d'entrée ?

Reformulation proposée : ajouter en règle métier : « Si le consultant n'a pas encore de CRA pour le mois visé, la première saisie le crée automatiquement en statut « Brouillon ». »

[À CLARIFIER] Le refus sur un week-end ou un jour férié n'a ni code ni message

La saisie est impossible sur un week-end ou un jour férié.

Ce qui manque : les notes techniques énumèrent 409 pour le jour déjà complet et 403 pour la mission inaccessible, et passent ce cas sous silence. On ne sait pas non plus si l'impossibilité est seulement une prévention d'interface — le jour n'est pas cliquable, ce que US-007 rend plausible puisqu'il grise déjà week-ends et fériés — ou un refus serveur, et dans ce cas quel message français l'utilisateur lit.

Question à poser : une requête `POST` portant sur un samedi ou un jour férié répond-elle 409 avec un message du type « Aucune saisie n'est possible sur un week-end ou un jour férié. » ?

Reformulation proposée : « La saisie est impossible sur un week-end ou un jour férié : le jour n'est pas cliquable dans le calendrier, et une requête portant sur un tel jour est refusée par le serveur avec un 409 et le message « Aucune saisie n'est possible sur un week-end ou un jour férié. » »

[À CLARIFIER] Le critère sur le CRA d'autrui n'est observable sur aucune route décrite

Un consultant ne peut pas saisir sur le CRA d'un autre consultant, contrôle fait côté serveur.

Ce qui manque : la route `POST /api/cra/{annee}/{mois}/entrees` ne porte aucun identifiant de consultant — le CRA visé est celui de l'appelant, résolu depuis l'en-tête `X-Demo-User` par `get_current_user` (`backend/app/core/deps.py`). Aucune requête ne peut donc désigner le CRA d'un autre : le critère est vrai par construction et le test d'acceptation correspondant n'a pas de sujet. Le cas réellement atteignable est ailleurs : `DELETE .../entrees/{id}` accepte l'identifiant d'une entrée qui peut appartenir à un autre consultant, et la story ne dit pas ce qui est renvoyé alors — 403 (règle de la story) ou 404 (convention de `NotFoundError` : « la ressource n'existe pas, ou n'est pas visible de l'appelant »).

Question à poser : supprimer une entrée appartenant à un autre consultant répond-il 404, comme une ressource non visible, ou 403 ?

Reformulation proposée : « Les entrées portent toujours sur le CRA de l'utilisateur courant, résolu côté serveur ; supprimer une entrée appartenant à un autre consultant est refusé avec un 404 et le message « Ressource introuvable. » »

[À CLARIFIER] Aucun état vide quand le consultant n'a aucune mission déclarable ce jour-là

Cliquer sur un jour ouvré propose les missions auxquelles je suis affecté et actives à cette date.

Ce qui manque : ce que voit le consultant quand cette liste est vide. Le cas est fréquent et déjà produit par le code : `list_available_missions ( backend/app/services/mission.py )` renvoie une liste vide dès que le consultant n'a aucune affectation, que ses missions sont clôturées, ou que leur période ne couvre pas la date. Aucun libellé français n'est fourni, et rien ne dit si le sélecteur s'ouvre quand même.

Question à poser : quel libellé français afficher lorsqu'aucune mission n'est déclarable à cette date ?

Reformulation proposée : ajouter un critère : « Si aucune mission n'est déclarable à cette date, le sélecteur affiche « Aucune mission déclarable à cette date. » et aucune saisie n'est possible sur ce jour. »

[À CLARIFIER] La saisie sur un jour futur n'est pas tranchée

Cliquer sur un jour ouvré propose les missions auxquelles je suis affecté et actives à cette date.

Ce qui manque : US-007 autorise explicitement la navigation « vers le futur, elle s'arrête au mois suivant ». Des jours ouvrés postérieurs à aujourd'hui sont donc affichés et cliquables, et rien ici ne dit s'ils sont saisissables. C'est une valeur limite, pas un détail : elle décide de la présence ou non d'un contrôle de date dans le service, et du message associé.

Question à poser : un consultant peut-il saisir sur un jour ouvré postérieur à la date du jour ?

Reformulation proposée : ajouter en règle métier la réponse retenue, par exemple : « La saisie est possible sur tout jour ouvré du mois affiché, passé comme futur — le calendrier ne propose de toute façon pas au-delà du mois suivant. »

[SUGGESTION] « le nom de la mission » désigne deux choses possibles

Le jour saisi affiche le nom de la mission et se distingue visuellement d'un jour vide.

Ce qui manque : un libellé de mission n'est unique que par client — la contrainte `UniqueConstraint("client", "label")` de `backend/app/models/mission.py` autorise deux clients à mener chacun une « Refonte SI ». Afficher le seul libellé peut donc rendre deux jours indistinguables. Par ailleurs « se distingue visuellement » n'est pas vérifiable mécaniquement, et la règle projet interdit de porter une information par la couleur seule.

Question à poser : le jour saisi affiche-t-il « client — libellé de la mission » ?

Reformulation proposée : « Le jour saisi affiche « client — libellé de la mission » ; il se distingue d'un jour vide par un fond coloré **et** par ce libellé, jamais par la couleur seule. »

[SUGGESTION] Le corps de la requête et celui de la réponse ne sont pas nommés

`POST /api/cra/{annee}/{mois}/entrees → 201 , DELETE /api/cra/{annee}/{mois}/entrees/{id} → 204 .`

Ce qui manque : les champs envoyés — le jour est-il une date complète ou un simple quantième, l'année et le mois étant déjà dans l'URL ? — et le contenu de la réponse `201` . Le quatrième critère suppose qu'un identifiant d'entrée est connu du client pour le `DELETE` : il faut donc que la création le renvoie, ce que la story ne dit pas.

Question à poser : la réponse `201` renvoie-t-elle l'entrée créée avec son identifiant, afin que le client puisse ensuite la supprimer ?

Reformulation proposée : compléter les notes techniques : « La requête porte le jour et l'identifiant de mission ; la réponse 201 renvoie l'entrée créée, identifiant compris. »

[SUGGESTION] Le prérequis portant la notion de mission « active » n'est pas déclaré

La mission doit être active à la date saisie et le consultant doit y être affecté.

Ce qui manque : « active à la date » recouvre deux notions issues de stories non déclarées en prérequis — le statut non clôturé (US-006) et la période de la mission (US-003) — alors que US-008 ne cite que US-005 et US-007. Le risque est nul en pratique, l'ordre de réalisation est numérique et le code tranche déjà (`list_available_missions` filtre sur `ACTIVE` et sur la période, en laissant passer une date de fin vide), mais le texte de la story gagnerait à le dire.

Question à poser : « active à la date » signifie-t-elle « mission non clôturée et dont la période couvre le jour saisi » ?

Reformulation proposée : « La mission doit être non clôturée et sa période doit couvrir la date saisie ; le consultant doit y être affecté. »

Prérequis

Les deux prérequis annoncés existent et sont réellement antérieurs.

- **US-005 (affectations)** couvre ce que la story suppose : le lien consultant ↔ mission, sans doublon possible, et sa suppression qui « n'efface aucune saisie d'activité déjà faite sur la mission » — cohérent avec le fait que US-008 crée ces saisies. Le modèle `Assignment` est en place et ne porte volontairement aucune période propre : ce sont les dates de la mission qui gouvernent.
- **US-007 (calendrier)** couvre l'écran, les jours ouvrés, les week-ends, les jours fériés et la route `GET /api/cra/{annee}/{mois}`. Elle ne couvre **pas** ce que US-008 suppose en plus : la persistance d'un CRA mensuel (premier constat ci-dessus), ni le fait qu'un jour ouvré soit cliquable.

Point non couvert par les prérequis déclarés mais déjà tranché ailleurs, donc sans risque : la liste des missions déclarables à une date existe déjà, service et route comprises (`GET /api/missions/disponibles`, `backend/app/routers/missions.py`).

Ce que la story établit pour la suite

- L'entrée de CRA elle-même — un jour, une mission, une journée complète — sur laquelle US-009 vient greffer le champ `fraction` et US-010 le champ `type`.
- Le couple de routes `POST .../entrees` et `DELETE .../entrees/{id}`, explicitement réutilisé tel quel par US-009 puis par US-010 : les codes de retour arrêtés ici ne seront plus rediscutés.
- La règle de propriété — une entrée appartient au CRA de son consultant, contrôle serveur — que US-012 étendra au statut du CRA (« le statut est revérifié côté serveur à chaque tentative de modification d'entrée »).
- La cellule de jour saisie dans le calendrier, support de l'affichage à deux missions de US-009 et des couleurs d'absence de US-010.
- Les entrées que US-011 comptera pour produire les totaux du mois, et sans lesquelles US-012 ne pourrait pas refuser la soumission d'un CRA vide.